

Course CS4720 Assignment Experimental Study

Genetic Algorithm

Punit Singh
200083296

Contents

Abstract	2
1 Solution 1: Single Crossover + Mutation - Recursive	2
1.1 Crossover	2
1.2 Mutation	3
1.3 Selection	3
2 Solution 2: Multiple candidate solutions - Elitism	5
2.1 Crossover	5
2.2 Mutation	6
2.3 Selection (Elitism)	6
3 Results	9

Abstract

In this report, I explain both of my approaches to solving the experimental study problem where we had to maximize the revenue generated by setting the prices of goods given an initial set of prices for 20 goods.

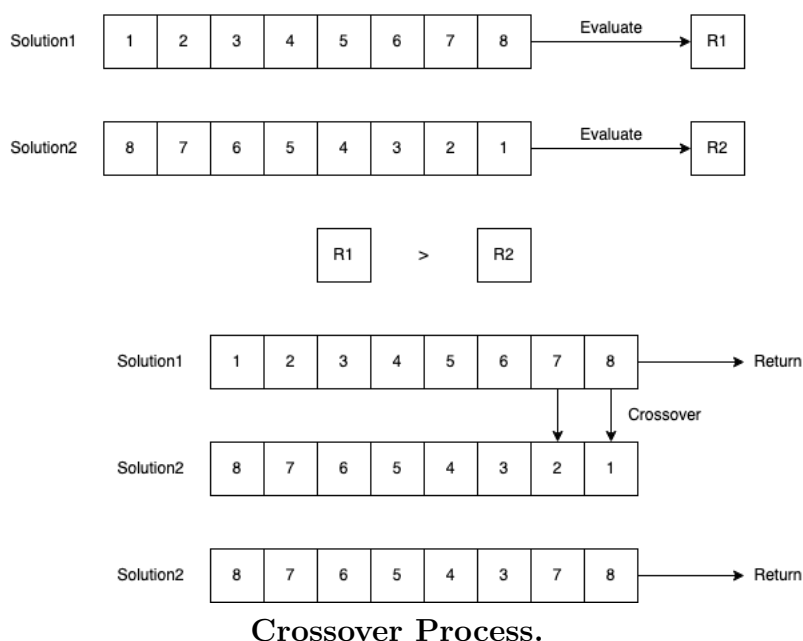
I used genetic algorithm as a base for both my solutions and developed two different architectures which although related to each other, are still quite different and produce different results. Although we were given only one set of prices, I also used a second set of prices which I generated from the given set itself.

1 Solution 1: Single Crossover + Mutation - Recursive

The first solution employs a simple genetic algorithm which follows the Algorithm [1]. We were given one randomly generated list of prices as the initial prices, but since genetic algorithm techniques like crossover require more than one candidate solutions, I generated a second set of prices by reversing the first array. I decided to reverse the original array because I did not want to introduce any external values into the prices list. After reversing, I now have two candidate solutions to begin with. I apply crossover, mutations and selection process on these two solutions.

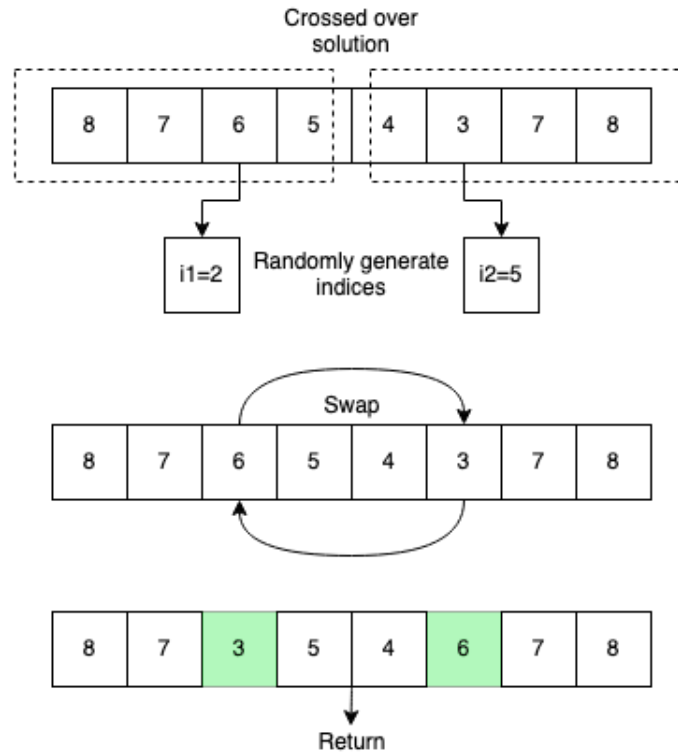
1.1 Crossover

For this solution, I set a crossover point of $length/5$. I evaluate the fitness of both the candidate solutions and compare the results and then cross the values of the solution with higher fitness over to the one with the lower fitness and continue on to the mutation process.



1.2 Mutation

The mutation process is fairly simple in that unlike some genetic algorithms, where mutation is done by flipping some bits, I am selecting 2 indices in each half of the array and swapping the values at both the indices mutation is carried out with a `MutationProbability` of 15%. A visual representation of the crossover technique that I am using is shown in image 1.2.



Mutation process.

1.3 Selection

The selection process in this solution is used to decide whether the candidate solutions generated after crossover and mutation are fit enough to be considered the next generation or not. Both of the candidate solutions are evaluated and their fitness recorded. If the fitness of any of the potential next generations is less than the fitness of the previous generation, then the `getNextGeneration` method is called again and the candidate solution goes through the process of crossover and mutation recursively until a candidate solution is reached whose fitness is greater than that of the previous generation.

The complete algorithm of this solution is given on the next page [1].

Algorithm 1 Pseudo code Solution 1

Require: $SOLUTION_1, SOLUTION_2 \leftarrow \text{reverse}(SOLUTION_1)$

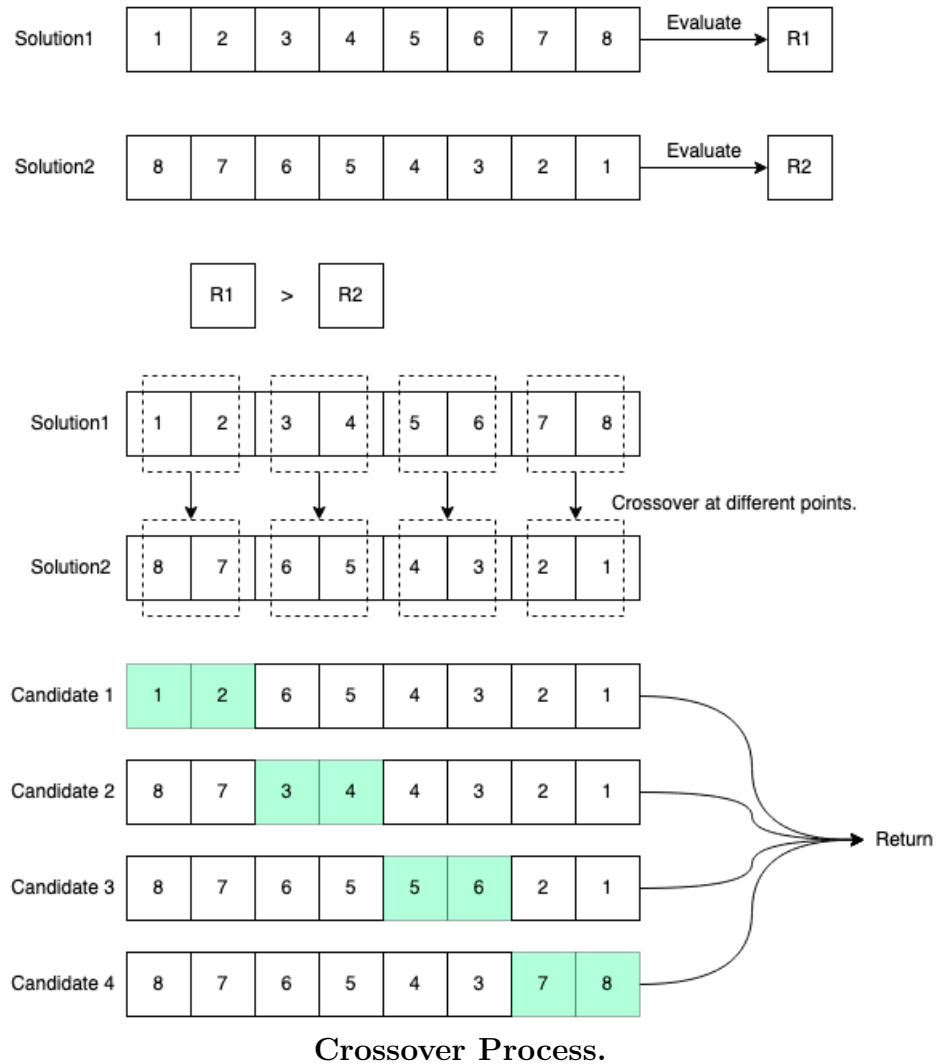
- 1: $REVENUE_1 \leftarrow \text{evaluate}(SOLUTION_1)$
- 2: $REVENUE_2 \leftarrow \text{evaluate}(SOLUTION_2)$
- 3: **if** $REVENUE_1 > REVENUE_2$ **then**
- 4: $BESTREVENUE \leftarrow REVENUE_1$
- 5: **else**
- 6: $BESTREVENUE \leftarrow REVENUE_2$
- 7: **end if**
- 8: **while** $iteration < maxIteration$ **do**
- 9: $REVENUE_1 \leftarrow \text{evaluate}(SOLUTION_1)$
- 10: $REVENUE_2 \leftarrow \text{evaluate}(SOLUTION_2)$
- 11: **if** $REVENUE_1 > REVENUE_2$ **then**
- 12: $CROSSOVER_2[15 : 20] \leftarrow SOLUTION_1[15 : 20]$
- 13: **else**
- 14: $CROSSOVER_1[15 : 20] \leftarrow SOLUTION_2[15 : 20]$
- 15: **end if**
- 16: $index_1 \leftarrow \text{randomInt}(0, 9)$
- 17: $index_2 \leftarrow \text{randomInt}(10, 19)$
- 18: $mutationProb \leftarrow \text{randomDouble}(0, 1)$
- 19: **while** $i < 2$ **do**
- 20: **if** $mutatioProb > 0.85$ **then**
- 21: $MUTATED_1 \leftarrow \text{swap}(CROSSOVER_1[index_1], CROSSOVER_1[index_2])$
- 22: $MUTATED_2 \leftarrow \text{swap}(CROSSOVER_2[index_1], CROSSOVER_2[index_2])$
- 23: **end if**
- 24: **end while**
- 25: $NEXTREV_1 \leftarrow \text{evaluate}(MUTATED_1)$
- 26: $NEXTREV_2 \leftarrow \text{evaluate}(MUTATED_2)$
- 27: **if** $NEXTREV_1 > REVENUE_1$ **then**
- 28: $SOLUTION_1 \leftarrow MUTATED_1$
- 29: **end if**
- 30: **if** $NEXTREV_2 > REVENUE_2$ **then**
- 31: $SOLUTION_2 \leftarrow MUTATED_2$
- 32: **end if**
- 33: **if** $REVENUE_1 > BESTREVENUE$ **then**
- 34: $BESTREVENUE \leftarrow REVENUE_1$
- 35: **end if**
- 36: **if** $REVENUE_2 > BESTREVENUE$ **then**
- 37: $BESTREVENUE \leftarrow REVENUE_2$
- 38: **end if**
- 39: **end while**
- 40: **return** $BESTREVENUE$

2 Solution 2: Multiple candidate solutions - Elitism

The architecture of this solution is quite similar to that of solution 1, but with some major changes which are discussed below. Here also I use two sets of initial prices as the starting point and again the second set again is generated by reversing the given set of prices. The difference is that after crossover, this algorithm generates 4 different candidate solutions followed by mutations on all of them only then to be tested and from among these four candidate solutions, only two are returned as the next generation prices.

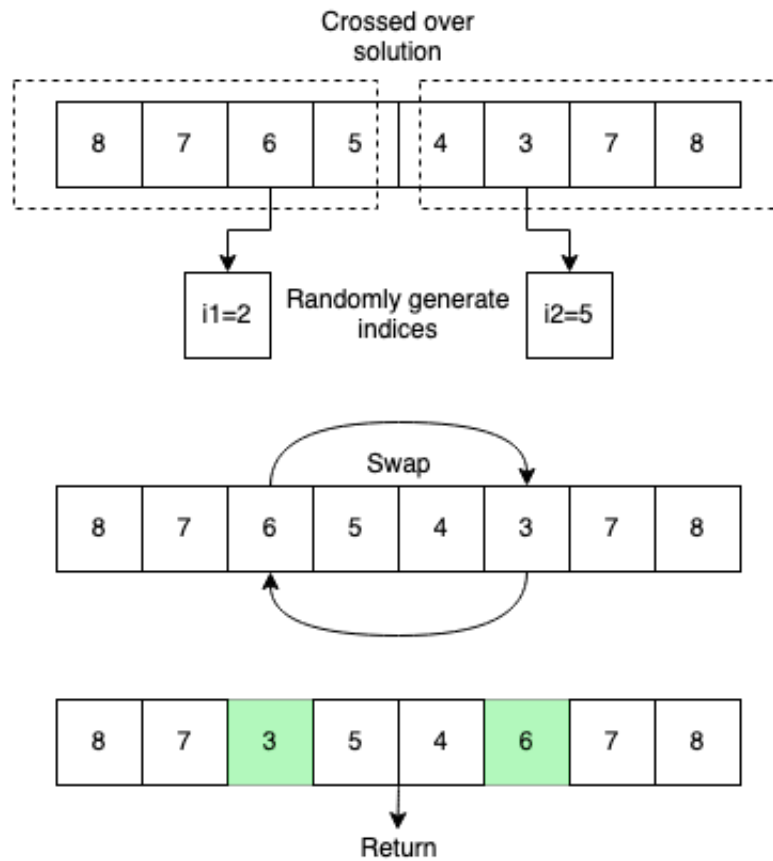
2.1 Crossover

In this solution, the crossover method used takes 2 arrays of prices as input and evaluates them. The solution with a higher fitness is then crossed over to the lower fitness solution at four crossover points, thus creating four different candidate solutions. A visual representation can be seen in image [2.1].



2.2 Mutation

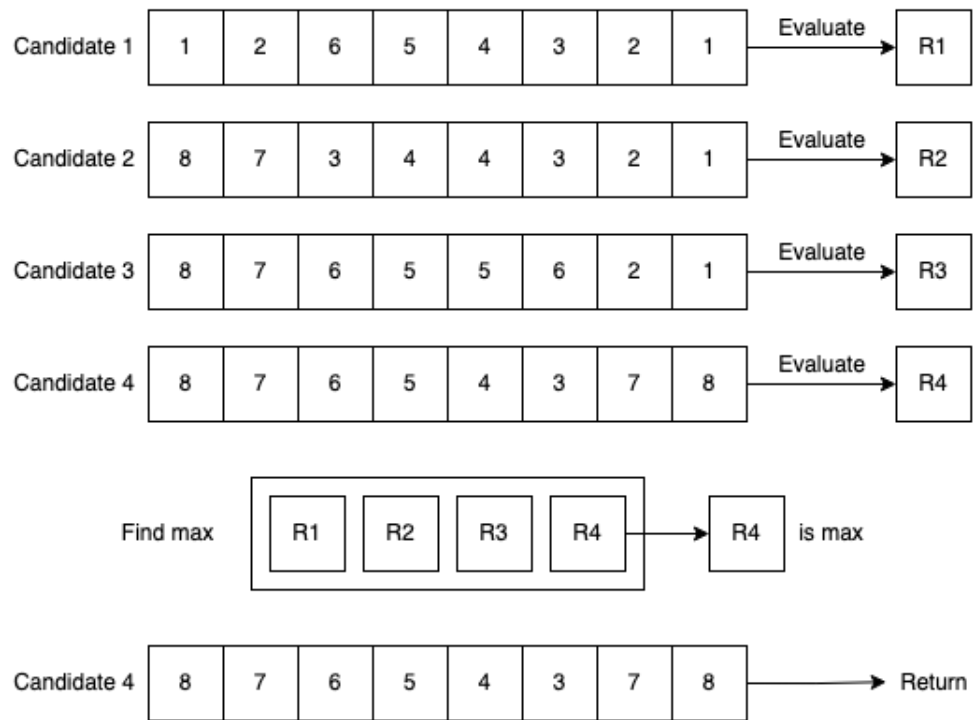
The mutation method used in this method is the same as that used in the previous solution. Although I did try to use a method of changing the values of randomly generated indices to a completely random value, the results of that experiments were varying too much with each run; performing extremely well at times and not good enough most of the other times. The same mutation method is applied to all four candidate solutions and the mutated candidates are forwarded to the selection method.



Mutation Process.

2.3 Selection (Elitism)

The selection method used in this solution is elitism where only the best two algorithms from the population of 4 candidate solutions are chosen as the next generation. This ensures that only the best traits are carried on to the next generation. A visual representation of the selection method for this solution is shown in image [2.3].



Selection Process.

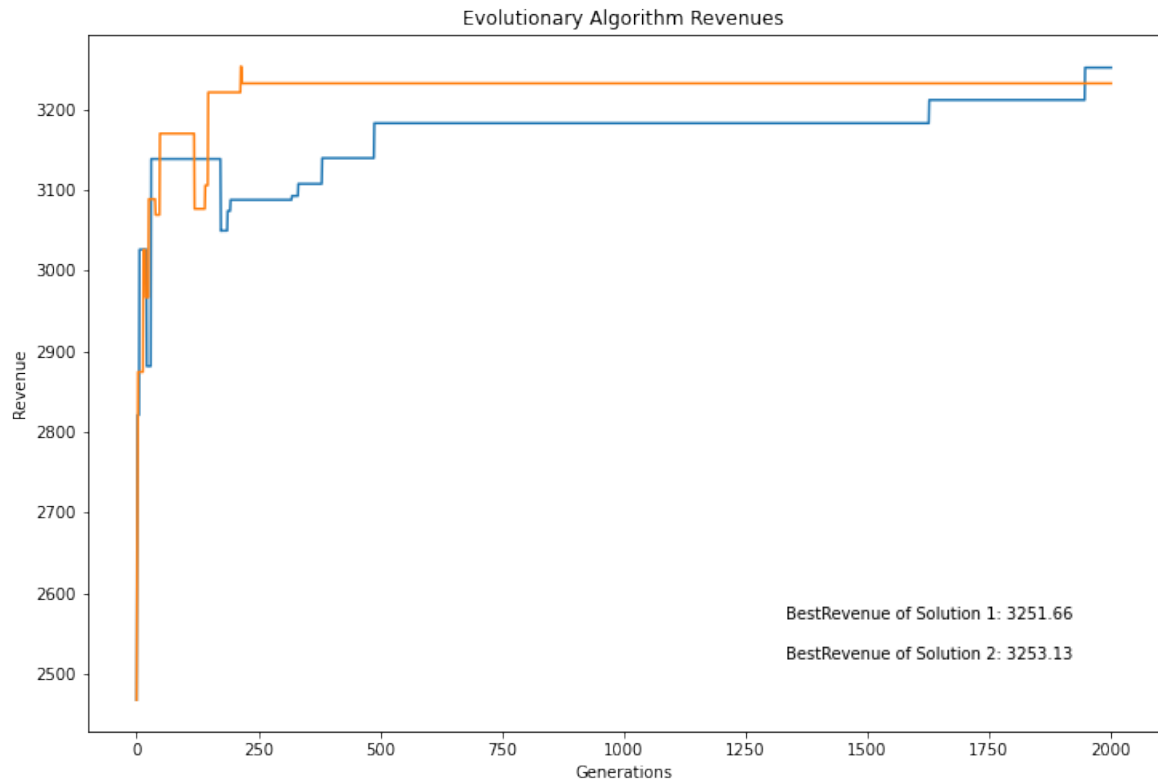
Algorithm 2 Pseudo code solution 2

Require: $SOLUTION_1, SOLUTION_2 \leftarrow \text{reverse}(SOLUTION_1)$

- 1: $REVENUE_1 \leftarrow \text{evaluate}(SOLUTION_1)$
- 2: $REVENUE_2 \leftarrow \text{evaluate}(SOLUTION_2)$
- 3: **if** $REVENUE_1 > REVENUE_2$ **then**
- 4: $BESTREVENUE \leftarrow REVENUE_1$
- 5: **else**
- 6: $BESTREVENUE \leftarrow REVENUE_2$
- 7: **end if**
- 8: **while** $iteration < maxIteration$ **do**
- 9: $REVENUE_1 \leftarrow \text{evaluate}(SOLUTION_1)$
- 10: $REVENUE_2 \leftarrow \text{evaluate}(SOLUTION_2)$
- 11: $BESTREVENUE \leftarrow \max(SOLUTION_1, SOLUTION_2)$
- 12: **if** $REVENUE_1 > REVENUE_2$ **then**
- 13: $CROSSOVER_1[0 : 4] \leftarrow SOLUTION_1[0 : 4]$
- 14: $CROSSOVER_2[5 : 9] \leftarrow SOLUTION_1[5 : 9]$
- 15: $CROSSOVER_3[10 : 14] \leftarrow SOLUTION_1[10 : 14]$
- 16: $CROSSOVER_4[15 : 19] \leftarrow SOLUTION_1[15 : 19]$
- 17: **else**
- 18: $CROSSOVER_1[0 : 4] \leftarrow SOLUTION_2[0 : 4]$
- 19: $CROSSOVER_2[5 : 9] \leftarrow SOLUTION_2[5 : 9]$
- 20: $CROSSOVER_3[10 : 14] \leftarrow SOLUTION_2[10 : 14]$
- 21: $CROSSOVER_4[15 : 19] \leftarrow SOLUTION_2[15 : 19]$
- 22: **end if**
- 23: $index_1 \leftarrow \text{randomInt}(0, 9)$
- 24: $index_2 \leftarrow \text{randomInt}(10, 19)$
- 25: $mutationProb \leftarrow \text{randomDouble}(0, 1)$
- 26: **while** $i < 2$ **do**
- 27: **if** $mutationProb > 0.85$ **then**
- 28: $MUTATED_{[1:4]} \leftarrow \text{swap}(CROSSOVER_1[index_1], CROSSOVER_1[index_2])$
- 29: **end if**
- 30: **end while**
- 31: $NEXTREV_{[1:4]} \leftarrow \text{evaluate}(MUTATED_1)$
- 32: $SOLUTION_1, SOLUTION_2 \leftarrow \text{getMaxPopulation}(NEXTREV_1, NEXTREV_2, NEXTREV_3, NEXTREV_4)$
- 33: **end while**
- 34: **return** $BESTREVENUE$

3 Results

After experimenting with different parameters and settings in both the solutions, I can conclude that both the algorithms give similar results. Although the first solution almost always performs slightly better than the second solution. After 2000 generations, the Best revenue generated by both the solutions is around 3000, which, I believe is a substantial improvement from the initial revenue of 2467.78.



Output plots of the solutions.